

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра цифровой трансформации (ЦТ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ

по дисциплине «Технологии проектирования и быстрой разработки приложений и сервисов»

Практическая работа №4

Магистрант группы ПТМО-01-25: Чугунова Д. И., Муромцева М.Д., Николаев А.С, Исаев В.В. _____ (подпись)

Преподаватель Снимщиков К. А. _____ (подпись)

Отчет представлен 05.10.2026

Москва 2026 г.

Оглавление

Введение . . . 3
Назначение работы . . . 3
Цель работы . . . 3
Задачи работы . . . 3
Проектирование интеграционных интерфейсов ProfIT . . . 4
Разграничение синхронного и асинхронного обмена . . . 4
Контуры доверия . . . 4
Трассировка C1/C2 к контрактам . . . 5
Границы реализации и проверки . . . 7
Синхронная спецификация OpenAPI . . . 8
Общие проектные решения контракта . . . 8
Реестр REST-операций . . . 8
Аутентификация доступа к попытке тестирования . . . 8
Ограничение частоты и ответ 429 . . . 9
Сквозная корреляция запросов . . . 9
Повторное создание попытки и статус 409 . . . 10
Среды, валюты, версии и доступность . . . 10
Асинхронная спецификация AsyncAPI . . . 12
Назначение, состав и статус документа . . . 12
Полезная нагрузка сообщений . . . 12
Конверт CloudEvents и корреляция . . . 13
TTL, DLQ, гарантии и политика повторов . . . 13
Выбор способа контроля сроков . . . 14
События для аналитики R-2–R-4 . . . 14
Взаимные ссылки OpenAPI ↔ AsyncAPI . . . 15
Проверка контрактов на локальном mock-сервере . . . 16
Используемые средства . . . 16
Подтверждённые результаты smoke-теста . . . 17
Негативные сценарии . . . 17
Открытые вопросы и дальнейшая работа . . . 18
Валидация спецификаций и воспроизводимость . . . 19
Заключение . . . 20
Список использованных источников . . . 20

Введение

Назначение работы

В рамках практической работы № 4 рассматривается проектирование и проверка интерфейсов информационной системы ProfIT, предназначенной для продвижения настольных игр и паззлов посредством офлайн-турниров, онлайн-тестирования и персональных промокодов. Система учитывает результаты активности, создает основания выдачи промокодов, передает письма получателям, проверяет применение скидок при покупке и формирует аналитические показатели.

Архитектурной основой решения является BPMSoft. Веб-приложение и мобильное приложение публичного пользователя обращаются к интеграционному API, внутренние сотрудники используют штатный клиент BPMSoft; торговый контур и почтовый провайдер представлены отдельными участниками взаимодействия. Архитектурные диаграммы C1 и C2 определяют направления обмена и границы ответственности.

В сдаваемый комплект входят две машинно-читаемые спецификации: `openapi.yaml` (OpenAPI 3.0.3) для синхронного HTTP-взаимодействия и `asyncapi.yaml` (AsyncAPI 3.0.0) для внутреннего обмена сообщениями. Настоящая редакция отчёта синхронизирована с актуальными YAML, скриптами проверки (`npm run check` / `npm run smoke`) и журналом `docs/validation_log.txt`.

Первая редакция отчёта описывала только REST-часть и фактически скрывала половину сданной работы (асинхронный контракт); кроме того, в самих YAML имелись дефекты (токен попытки не был объявлен схемой безопасности, `429` без `Retry-After`, отсутствовали TTL/DLQ). В актуальной редакции все выявленные дефекты устранены — ниже разобрано, что изменено в спецификациях и как это проверено.

Репозиторий учебного проекта: https://github.com/ChaoticPost/BPMSoft. Технические выводы основаны на локальной проверке файлов репозитория (12 REST-операций, 6 каналов AsyncAPI, smoke с позитивными и негативными сценариями).

Цель работы

Цель работы — спроектировать и документировать синхронные и асинхронные интерфейсы ProfIT, установить соответствие контрактов архитектуре C1/C2, доменной модели и функциональному бэклогу E-1–E-7, а также оценить качество спецификаций и результаты их локальной проверки.

Задачи работы

- Определить участников, направления передачи данных и границы доверия по C1/C2.
- Описать назначение OpenAPI и AsyncAPI и выявить состав соответствующих операций и сообщений.
- Проверить соответствие полей, статусов, ошибок и переходов доменной модели ProfIT.
- Обосновать аутентификацию публичных попыток тестирования и доверенных интеграций.
- Зафиксировать требования к ограничению частоты, корреляции, идемпотентности и жизненному циклу сообщений.
- Описать выполненные испытания Prism, негативные сценарии и ограничения mock-проверки.

- Зафиксировать открытые вопросы и границы доказанного.
- Синхронизировать описание в отчёте с актуальными YAML репозитория.

Проектирование интеграционных интерфейсов ProfIT

Разграничение синхронного и асинхронного обмена

OpenAPI 3.0.3 применяется для взаимодействия клиента с интеграционным API по HTTP/JSON: просмотр каталога, прохождение онлайн-тестирования, проверка промокода торговым контуром, передача подтверждения покупки, импорт продаж и прием почтового webhook. Для интерфейсов заданы параметры, статусы, аутентификация, ошибки и идемпотентность.

AsyncAPI 3.0.0 описывает внутренние асинхронные задачи и события. В актуальном `asyncapi.yaml`: операции `send` / `receive`, конверт CloudEvents 1.0, расширение `correlationid`, версионированные имена каналов (`*.v1`), команда повторной отправки, идентификатор события для дедупликации (`source + id`), durable-очереди, **предложенные** значения TTL (`x-message-ttl`) и DLX/DLQ (`x-dead-letter-*`) в штатном поле `queue.arguments` AMQP-bindings, а также отдельный канал `promocodes.dlx` для недоставленных сообщений. Почтовый webhook остаётся внешней HTTP-точкой OpenAPI и не смешивается с AMQP.

Граница «синхронно/асинхронно» проведена по инициатору и надёжности доставки: всё, что ждёт ответа внешний участник (клиент, торговый контур, провайдер), остаётся HTTP; всё, что выполняется фоном внутри контура (постановка отправки, повтор, отложенные проверки сроков, факт активации), уходит в брокер. Администрирование в штатном клиенте BPMSoft и аналитика Insight в заказной контракт не дублируются.

Контуры доверия

Контур	**Участник и права**	**Аутентификация**
Public	Анонимный посетитель; опубликованные данные и собственная попытка теста	`AttemptTokenAuth` (3 операции); остальное открыто
Commerce	Внешний торговый контур; validate, purchases, sales	`CommerceApiKey` (`X-API-Key`)
MailProvider	Почтовый провайдер; только webhook статусов	`MailWebhookSignature` (`X-Provider-Signature`)

Таблица 1. Контуры доверия и схемы безопасности OpenAPI.

Анонимное создание попытки — осознанное исключение из правила «всё защищено»: токен выдаётся в ответе и связывает клиента только с его попыткой. Служебная аутентификация сотрудников BPMSoft в контракт не входит — доверенные операции ограничены интеграционным фасадом с минимальными правами.

Трассировка C1/C2 к контрактам

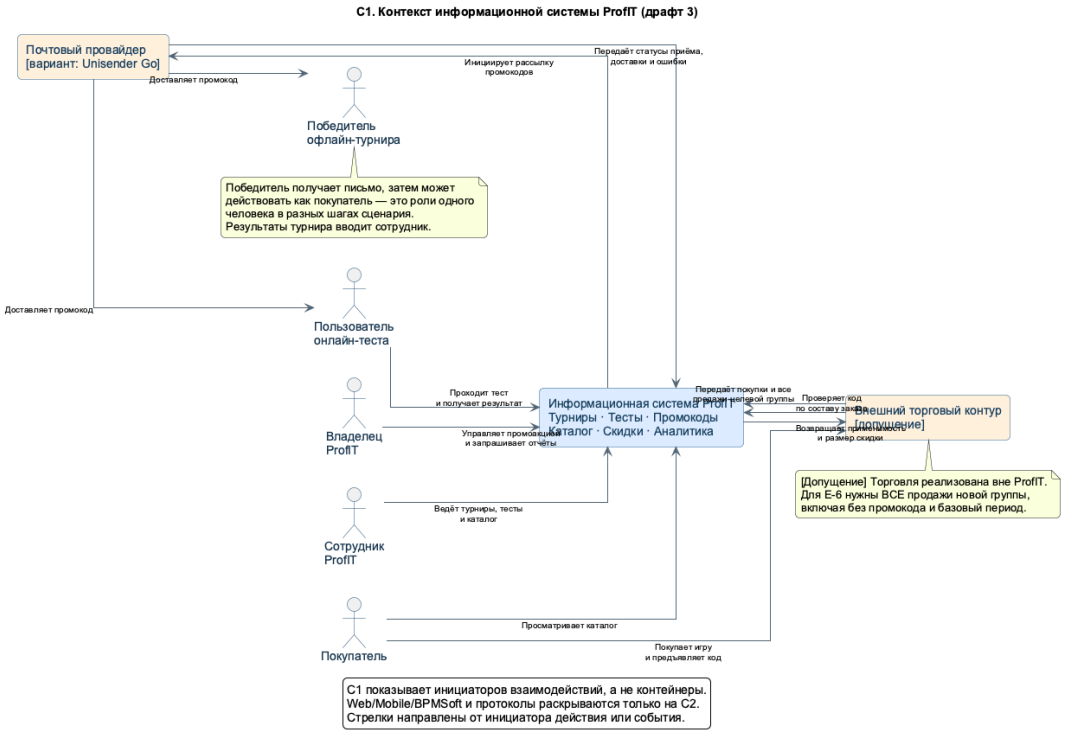


Рис. 1. C1 — контекст информационной системы ProfIT (драфт 3).

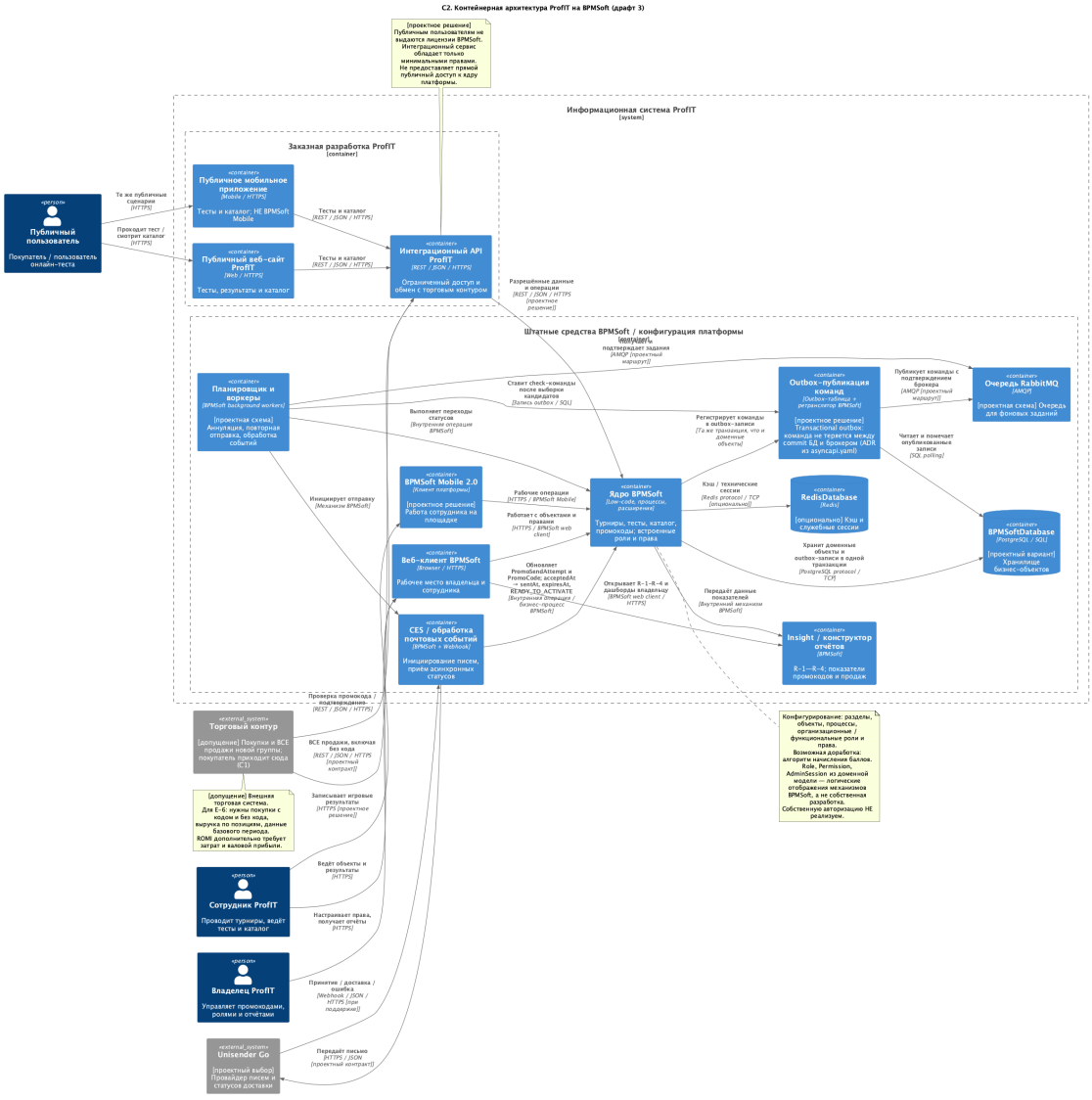


Рис. 2. C2 — контейнерная архитектура ProfIT на BPMSoft (драфт 3): контейнер «Outbox-публикация команд» реализует ADR transactional outbox из `asyncapi.yaml`.

Сценарий	Архитектурный маршрут	Контракт	Эпик
Просмотр каталога	Web/Mobile → интеграционный API → ядро BPMSoft	OpenAPI: публичный GET	E-4
Прохождение теста	Web/Mobile → API → ядро BPMSoft	OpenAPI: попытка, ответы, результат	E-2
Применение скидки	Торговый контур → API → ядро BPMSoft	OpenAPI: проверка и покупка	E-5
Все продажи	Торговый контур → API → BPMSoft/Insight	OpenAPI: данные продаж	E-6
Отправка промокода	BPMSoft → задания/воркеры → CES → провайдер	AsyncAPI: команды; внешний HTTP	E-3
Статус письма	Провайдер → webhook/CES → ядро BPMSoft	OpenAPI: webhook; внутренняя обработка	E-3
Контроль сроков	Планировщик → check-команды → объекты BPMSoft	AsyncAPI: timeout/expire check	E-3
Отчетность владельца	Владелец → веб-клиент BPMSoft → Insight	Штатный доступ; не заказной REST	E-6

Таблица 2. Трассировка сценариев C1/C2 к контрактам и эпикам.

Каждому публичному сценарию C1/C2 сопоставлена операция или сообщение одной из двух спецификаций; единственный поток без заказного контракта — штатный доступ владельца к отчётности Insight, что осознанно: дублировать административный API не требуется.

Границы реализации и проверки

Контракт описывает ожидаемое поведение интеграционных границ, но не доказывает, что бизнес-правила исполняются в работающей системе BPMSoft. Запуск Prism демонстрирует доступность mock-маршрутов и согласованность выбранных примеров запросов с описанными ответами. Он не подтверждает атомарность активации промокода, фактическую отправку писем, сохранение в базе, соблюдение rate limit на живом сервисе или работу RabbitMQ.

Доказанному и заявленному в отчёте всюду противопоставлено явно: «в контракте объявлено» — значит присутствует в YAML и валидируется парсерами; «исполнено в smoke» — значит mock вернул ожидаемый код и форму ответа; «предложение для согласования» — значит значение требует утверждения и не считается проверенным.

Синхронная спецификация OpenAPI

Общие проектные решения контракта

До реестра операций зафиксированы сквозные соглашения: ошибки возвращаются как `application/problem+json` по RFC 9457 с целочисленным `status` и машинными кодами в `violations`; тела успешных ответов завёрнуты в `data` (+ `meta.pagination` у списков); идентификаторы — UUID; денежные суммы — строка decimal scale=2 без float; в `validate` цена из запроса магазина считается справочной (`unitPriceQuoted`), а скидка рассчитывается по авторитетным данным каталога (`unitPriceAuthoritative`) — это продолжает инвариант доменной модели о позиционных скидках. Активация промокода выполняется однократно и только при покупке с кодом в `READY_TO_ACTIVATE`; публичная анонимная активация запрещена, код промокода публично не возвращается.

Реестр REST-операций

В `openapi.yaml` зарегистрированы следующие маршруты (12 операций). Таблица отражает состав опубликованного контракта.

Метод / маршрут	**Назначение**	**Контур**
`GET /public/products`	Получение опубликованных товаров	Публичный
`GET /public/products/{productId}`	Карточка товара	Публичный
`GET /public/tests`	Доступные опубликованные тесты	Публичный
`GET /public/tests/{testId}/published-version`	Опубликованная версия теста	Публичный
`POST /public/test-attempts`	Создание тестовой попытки	Публичный
`GET /public/test-attempts/{attemptId}`	Получение собственной попытки	`AttemptTokenAuth`
`POST /public/test-attempts/{attemptId}/answers`	Отправка ответа	`AttemptTokenAuth`
`POST /public/test-attempts/{attemptId}/finish`	Завершение и расчет результата	`AttemptTokenAuth`
`POST /integrations/commerce/promocodes/validate`	Проверка скидки по составу заказа	Торговый `X-API-Key`
`POST /integrations/commerce/purchases`	Подтверждение покупки и идемпотентная активация	Торговый
`POST /integrations/commerce/sales`	Передача всех продаж товарной группы	Торговый
`POST /integrations/mail/provider-events`	Получение статуса письма	Webhook-подпись

Таблица 3. Реестр REST-операций OpenAPI.

Аутентификация доступа к попытке тестирования

В первой редакции контракта `X-Attempt-Token` был объявлен обычным header-параметром, а у операций стоял пустой `security` — то есть «аутентификация не требуется» при заявленных ответах `401` и `403`. Спецификация противоречила сама себе: генераторы клиентов оставляли токен произвольной строкой в сигнатуре, инструменты документирования не показывали признак защищённости, а анализаторы безопасности считали операции открытыми.

В актуальном контракте схема `AttemptTokenAuth` (`type: apiKey`, `in: header`, `name: X-Attempt-Token`) объявлена в `components.securitySchemes` и подключена через `security` у трёх операций: получение попытки, отправка ответа, завершение. Теперь токен трактуется как учётные данные, а коды `401`/`403` имеют обоснование. Создание попытки остаётся открытым (`security: []`): токен выдаётся в ответе `201` (`data.attemptToken` + `attemptTokenExpiresAt`) и далее обязателен — защита без обязательной регистрации сохраняется.

Заголовок не передаётся в URL и не попадает в публичные логи. Проверка принадлежности токена попытке и срока его действия — серверная ответственность (mock Prism её не исполняет). Фрагмент контракта:

```
components:
  securitySchemes:
    AttemptTokenAuth:
      type: apiKey
      in: header
      name: X-Attempt-Token
# у трёх операций попытки (get/answers/finish):
security:
  - AttemptTokenAuth: []
```

Ограничение частоты и ответ 429

Ранее ответ `429` был объявлен без указания времени повтора: клиент получал отказ и не знал, когда обращаться снова. В актуальном контракте общий ответ `TooManyRequests` включает заголовок `Retry-After` (секунды до допустимого повтора, RFC 9110; допустим и HTTP-date) и пример `detail` в `application/problem+json`. `429` объявлен у публичных чтений каталога и тестов, создания попытки и всех интеграционных операций — везде, где лимит осмыслен.

Специализированные заголовки `RateLimit-Limit` / `RateLimit-Remaining` / `RateLimit-Reset` сознательно не вводятся: пока реализация лимитера не существует, контракт ограничивается единым обязательным соглашением `Retry-After`. Тот же заголовок объявлен и у `503`.

```
responses:
  TooManyRequests:
    description: Превышен лимит запросов. Повторять запрос не раньше срока,
заданного заголовком `Retry-After`.
    headers:
      Retry-After:
        $ref: '#/components/headers/RetryAfter'
      X-Correlation-ID:
        $ref: '#/components/headers/CorrelationId'
```

Сквозная корреляция запросов

До правки идентификатор корреляции существовал только на стороне ответа — в теле ошибки и в конверте события, поэтому сквозная трассировка «публичный клиент → интеграционный API → ядро → очередь» фактически начиналась с ответа. В актуальном контракте входящий заголовок описан явно: reusable-параметр `CorrelationId` (`X-Correlation-ID`, optional, 8–128 символов) подключён ко всем 12 операциям.

Если клиент не передаёт заголовок, сервер генерирует UUID сам. Значение эхом возвращается в заголовке `X-Correlation-ID` всех успешных ответов и ответов `429`/`500`/`503`, а в телах `application/problem+json` дублируется полем `correlationId`. Дальше тот же идентификатор уходит в брокер расширением `correlationid` конверта CloudEvents — цепочка становится сквозной. Корреляция не является секретом и не заменяет проверку авторизации.

```
parameters:
  CorrelationId:
    name: X-Correlation-ID
    in: header
    required: false
    description: Опциональный id корреляции. Если отсутствует —
      сервер генерирует UUID и возвращает в заголовке ответа.
    schema: { type: string, minLength: 8, maxLength: 128 }
```

Повторное создание попытки и статус 409

Замечание о неограниченном анонимном создании попыток закрыто двумя видимыми в контракте механизмами. Первый — бизнес-ограничение, записанное в `info.description` и в описании `createTestAttempt`: не более одной незавершённой попытки на пару (`publishedVersionId`, нормализованный email), если email уже известен — передан при создании (`recipientEmail`, опционально) или зафиксирован предыдущим `finish`. Нарушение → `409` с машинным кодом `TEST_ATTEMPT_IN_PROGRESS`. Второй — rate limit на создание попыток по IP/клиенту → `429` + `Retry-After`: он сдерживает обход ограничения через разные адреса до того, как email становится известен.

Отдельный `Idempotency-Key` на `POST /public/test-attempts` осознанно не вводится: конфликт активной попытки закрывается `409`, частота — `429`. Обязательный ключ объявлен для остальных операций с побочным эффектом (answers, finish, purchases, sales), опциональный — для validate и mail webhook. Правило повтора едино: тот же ключ и то же тело → исходный успешный ответ без повторного эффекта; тот же ключ с другим телом → `409` (`IDEMPOTENCY_KEY_BODY_MISMATCH`); срок хранения ключа — не менее 24 ч (предложение для согласования).

Прочие основания `409` перечислены в общем ответе `Conflict`: `ANSWER_ALREADY_SUBMITTED`, `ATTEMPT_ALREADY_FINISHED`, дубликат `purchaseExternalId` и дубликат `providerEventId`. Продуктовый вопрос «сколько раз вообще разрешено проходить один тест» остаётся открытым — контракт фиксирует лишь проверяемое ограничение, не подменяя продуктовое правило.

Среды, валюты, версии и доступность

- **Среды.** В `servers` объявлены два окружения — `profit-dev` (условная среда разработки) и `profit-test` (условный тестовый стенд интеграций); оба помечены «не production», адреса настоящего стенда не выдумываются. Разделение сред теперь видно прямо в контракте, а не только в тексте.
- **Валюта.** `CurrencyCode` — enum из одного значения `RUB`: осознанное сужение MVP. В описании схемы и `info.description` зафиксировано, что добавление валют — ломающее расширение: новая major-версия API (`/api/v2` или согласованный feature-flag),

предупреждение потребителей минимум за 30 дней, поддержка `v1` до вывода по согласованному календарю.

- **Версионирование и вывод из эксплуатации.** Префикс `/v1` дополнен политикой: совместимые изменения (новые optional-поля, новые документированные коды в `reasons`) идут внутри `v1`; удаление или переименование полей, смена типа, ужесточение обязательности и изменение семантики статусов считаются ломающими и требуют новой major URL-версии. Вывод версии — deprecation-объявление в `info`/`servers`, предложенный `Sunset`-заголовок и параллельная поддержка не менее согласованного с потребителями срока.
- **Доступность.** `503 Service Unavailable` применён единообразно ко всем 12 операциям (раньше компонент был объявлен, но использовался в одной операции), при недоступности ядра BPMSoft или зависимостей; при наличии — `Retry-After`. Замечание о «висячем» общем компоненте снято.

Асинхронная спецификация AsyncAPI

Назначение, состав и статус документа

asyncapi.yaml (AsyncAPI 3.0.0) описывает команды и события контура BPMSoft/CES: брокер — RabbitMQ (servers.rabbitmqProject, AMQP, virtual host /, userPassword-аутентификация как проектное решение). Webhook провайдера — внешняя HTTP-точка OpenAPI POST /integrations/mail/provider-events и намеренно не смешивается с AMQP.

В файле 6 каналов и 12 операций (send/receive — по паре на канал; этим заданы отправитель и получатель каждого потока):

Канал	**Производитель → потребитель**	**Назначение**
promocodes.dispatch.requested.v1	BPMSoft → воркер/CES	Первичная отправка (outbox)
promocodes.dispatch.retry.requested.v1	BPMSoft/планировщик → воркер	Повтор без нового PromoCode
promocodes.dispatch.timeout.check.v1	Планировщик → BPMSoft	Аннуляция по deliveryDeadlineAt
promocodes.validity.expire.check.v1	Планировщик → BPMSoft	Аннуляция по expiresAt
promocodes.activated.v1	BPMSoft → подписчики	Событие активации (outbox)
promocodes.dlx	RabbitMQ → операционный разбор	Недоставленные сообщения (очереди *.dlq)

Таблица 4. Каналы AsyncAPI (актуальный YAML).

Статус проверки. Спецификация валидна по парсеру @asyncapi/parser 3.6.5 (см. docs/validation_log.txt): каналы, операции, сообщения и схемы разбираются без ошибок. Для каждого сообщения приведён пример payload, соответствующий объявленной схеме. Публикации в реальный брокер не выполнялось — гарантии ниже описаны как требования контракта, а не как проверенное поведение инфраструктуры.

Полезная нагрузка сообщений

Сообщение	**Обязательные поля data**
PromoDispatchRequested	promoCodeId, ground (GAME/TEST), productGroupId, validityDays, deliveryDeadlineAt, recipientEmail, attemptNumber = 1
PromoDispatchRetryRequested	promoCodeId, attemptNumber ≥ 2, reason (PREVIOUS_ATTEMPT_FAILED / MANUAL), recipientEmail, deliveryDeadlineAt
PromoDispatchTimeoutCheck	promoCodeId, deliveryDeadlineAt, expectedAnnulReason = DISPATCH_TIMEOUT
PromoValidityExpireCheck	promoCodeId, expiresAt, expectedAnnulReason = VALIDITY_EXPIRED
PromoCodeActivated	promoCodeId, purchaseId, purchaseExternalId, activatedAt, productGroupId, discountTotal

Таблица 5. Схемы полезной нагрузки сообщений AsyncAPI.

expectedAnnulReason в check-сообщениях фиксирует ожидаемую причину аннуляции и делает проверку самодокументируемой: воркер переводит код в ANNULLED только если

условие срока ещё актуально на момент обработки. `attemptNumber` = 1 для первой отправки и ≥ 2 для retry — бизнес-ключ идемпотентности потребителя.

Конверт CloudEvents и корреляция

Структурированный режим CloudEvents 1.0: `specversion` (`'1.0'`), `id` (уникален в пределах `source`), `source` (URI-производителя, напр. `urn:profit:bpmsoft-core`), `type` (соответствует каналу с суффиксом версии `.v1`), `subject` (обычно `promoCodeId`), `time`, `datacontenttype` (`application/json`), `data`. Корреляция передаётся расширением `correlationid` в конверте и AMQP-заголовком `correlation-id` (общий messageTrait `cloudEventHeaders`) — этим сквозная корреляция связывается с REST `X-Correlation-ID`.

Уникальность при повторной доставке определяется парой `source + id` — тот же принцип, что и дедупликация webhook по `providerEventId`. Бизнес-идентификаторы (`promoCodeId`, `sendAttemptId`, `attemptNumber`) связывают сообщение с объектами `PromoCode` и `PromoSendAttempt`. PII-поле `recipientEmail` есть только в командах отправки (потребители — доверенные воркеры); в событии активации email намеренно отсутствует — он не нужен подписчикам.

TTL, DLQ, гарантии и политика повторов

Замечание об отсутствии времени жизни и очереди недоставленных сообщений закрыто в AMQP-bindings: у четырёх командных очередей в штатном поле `queue.arguments` (аргументы `queue.declare`) заданы `x-message-ttl`, `x-dead-letter-exchange = promocodes.dlx` и `x-dead-letter-routing-key = <канал>.dlq`. Значения помечены в YAML как предложение для согласования:

Канал	TTL	Обоснование
`dispatch.requested` / `retry.requested`	24 h	не больше типичного окна `deliveryDeadlineAt`
`dispatch.timeout.check`	1 h	разовая проверка после выборки планировщика
`validity.expire.check`	1 h	аналогично
`promocodes.activated` (exchange)	72 h	допуск отстающих подписчиков; зафиксировано в описании канала

Таблица 6. Предлагаемые TTL сообщений по каналам.

Профиль доставки: `at-least-once`, ACK только после успешной бизнес-операции или сохранения результата, идемпотентный потребитель по бизнес-ключу (`promoCodeId` + `attemptNumber` для отправок, `promoCodeId` + тип check для проверок). Exactly-once end-to-end не утверждается. Временные технические ошибки повторяются воркером с растущей задержкой (предложение: max 5, backoff 1m/5m/15m/1h/6h); после исчерпания повторов сообщение маршрутизируется в DLQ с исходным `id` и причиной, предлагаемое хранение DLQ — 7 суток для разбора. Это разрывает бесконечный цикл неразбираемых сообщений. После наступления `deliveryDeadlineAt` новая отправка не инициируется, даже если сообщение физически ещё присутствует в очереди.

Пример привязки командной очереди (`promocodes.dispatch.requested.v1`):

```
bindings:
  amqp:
    is: queue
```

```yaml
    queue:
      name: promocodes.dispatch.requested.v1
      durable: true
      arguments:
        x-message-ttl: 86400000
        x-dead-letter-exchange: promocodes.dlx
        x-dead-letter-routing-key: promocodes.dispatch.requested.v1.dlq
```

`queue.arguments` — штатное поле привязки канала AMQP 0-9-1 для аргументов `queue.declare`. Значения предназначены для ручного или скриптового создания очередей (infrastructure provisioning до согласования инфраструктурного конфига); генераторы кода и средства автоматического развёртывания из контракта их не применяют — в YAML это отмечено комментарием у каждой очереди. Факт записи в YAML не равен применению параметров на реальном брокере: развёртывание RabbitMQ в рамках практики не выполнялось.

Сама очередь недоставленных сообщений описана в контракте наравне с остальными — каналом `promocodes.dlx` (direct-обменник и durable-очереди `*.dlq` за ним). В него попадают исходные конверты команд без изменения полезной нагрузки: брокер дополняет их AMQP-заголовками `x-death` и `x-first-death-*` с причиной и очередью первого недоставления. Потребитель канала определён явно — это операционная процедура разбора (мониторинг глубины `*.dlq`, алерт дежурному, ручной или скриптовый повтор через ядро по состоянию БД), а не автоматический воркер: автоматическое чтение DLQ контрактом сознательно не объявляется, чтобы исключить цикл «DLQ → повтор → DLQ».

Выбор способа контроля сроков

Способ контроля сроков зафиксирован как осознанное проектное решение (ADR в `info.description` `asyncapi.yaml`):

1. Первая отправка и retry — **команда на каждый PromoCode** через transactional outbox: гарантирует постановку без потери между commit БД и брокером. На контейнерной диаграмме C2 (драфт 3) этот механизм представлен отдельным контейнером «Outbox-публикация команд» (outbox-таблица в общей БД + ретранслятор BPMSoft): ядро и планировщик пишут доменные объекты и outbox-записи в одной транзакции, а публикует их в RabbitMQ только ретранслятор с подтверждением брокера — контракт и архитектура описывают одно и то же решение.
2. `DISPATCH_TIMEOUT` / `VALIDITY_EXPIRED` — **периодическая выборка** планировщика по БД (кандидаты с истёкшим `deliveryDeadlineAt` / `expiresAt`) с публикацией check-команд только для найденных id.

Отклонённая альтернатива — таймерная команда на каждый промокод в момент создания: она давала бы поток сообщений, пропорциональный числу выданных кодов, дублировала бы дедлайны, уже хранящиеся в БД, и усложняла отмену при ручной аннуляции. Частота опроса планировщика и критерий просрочки — открытые вопросы для согласования.

События для аналитики R-2–R-4

Событие активации (`promocodes.activated.v1`) есть; отдельное потоковое событие первой успешной отправки в MVP **не** вводится — решение принято явно: когорты выдачи для

отчётов R-2–R-4 строятся по полю `sentAt` (и связанным статусам) в основной БД через Insight, а webhook `ACCEPTED_BY_PROVIDER` обновляет БД. Если позже появится потоковая аналитика вне БД, потребуется типизированное событие первой допустимой отправки с надёжной публикацией через outbox — это зафиксировано как открытый вопрос владельцу отчётности.

Взаимные ссылки OpenAPI ↔ AsyncAPI

В обоих документах заданы `externalDocs` с указанием парного контракта и пояснением, что webhook почты — HTTP (`openapi.yaml`), а фоновые команды — `asyncapi.yaml`. Относительные имена файлов в репозитории: корневые `openapi.yaml` и `asyncapi.yaml`.

Проверка контрактов на локальном mock-сервере

Используемые средства

Стек: Node.js/npm, `@stoplight/prism-cli`, сценарий `scripts/smoke-mock.js`, валидаторы `@apidevtools/swagger-parser`, `@asyncapi/parser` и линтер `@redocly/cli`.

Команды:

```
npm install
npm run check           # парсеры → docs/validation_log.txt
npm run lint:openapi    # redocly lint openapi.yaml
npm run mock            # prism mock openapi.yaml -p 4010 -h 127.0.0.1
npm run smoke           # scripts/smoke-mock.js
# либо одной командой:
npm run evidence
```

По журналу `docs/validation_log.txt` (прогон 2026-11-05):

- Node.js `v24.18.0`, npm `11.16.0`, Prism `5.16.0`, `swagger-parser 12.1.0`, `@asyncapi/parser 3.6.5`
- OpenAPI OK — 3.0.3, paths=12, ops=12; securitySchemes: `CommerceApiKey`, `MailWebhookSignature`, `AttemptTokenAuth`; servers: `profit-dev`, `profit-test`
- AsyncAPI OK — 3.0.0, channels=6, ops=12
- `redocly lint openapi.yaml` — спецификация валидна, без предупреждений
- Smoke завершился сообщением `Smoke OK` (6 позитивных + 8 негативных проверок)
- В лог записаны `git HEAD` и отпечаток дерева (`git write-tree`) — ревизия проверенных файлов восстанавливается однозначно

Скриншоты Prism (если приложены к сдаче) размещаются в `docs/report/images/` как `image1.png` … `image2.1.png`. Основное машинное доказательство — `docs/validation_log.txt`.

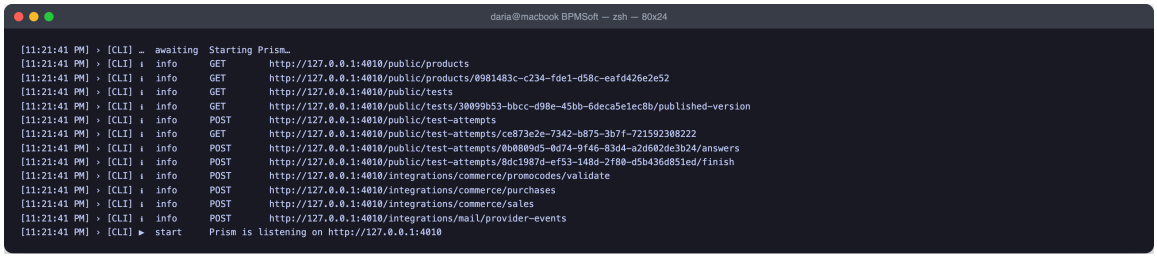


Рис. 3. Запуск Prism и регистрация REST-маршрутов.

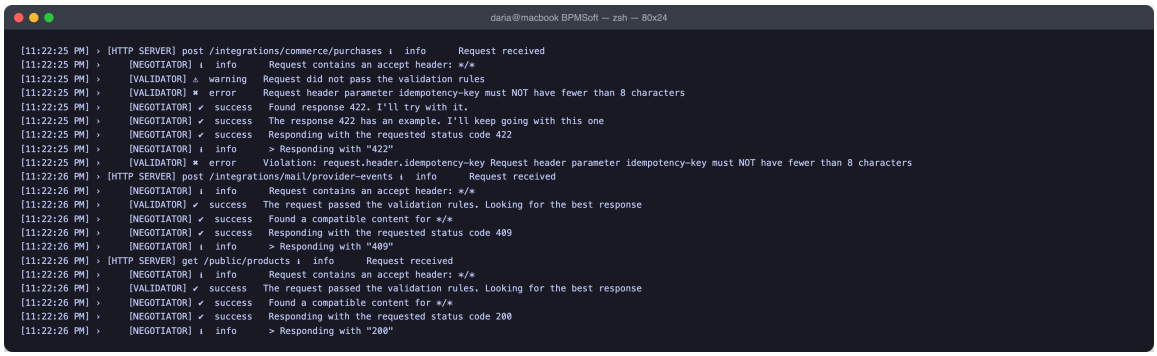


Рис. 4. Журнал валидации и формирования HTTP-ответов Prism.

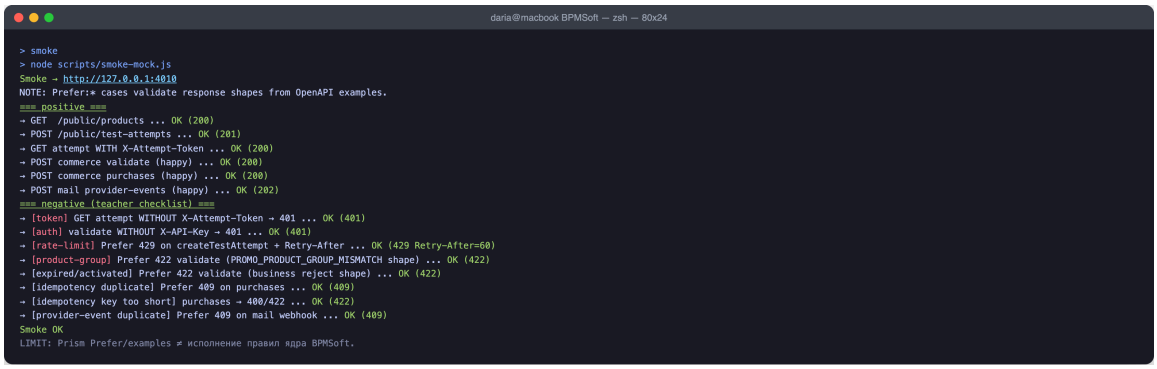


Рис. 5. Завершение основного smoke-сценария.

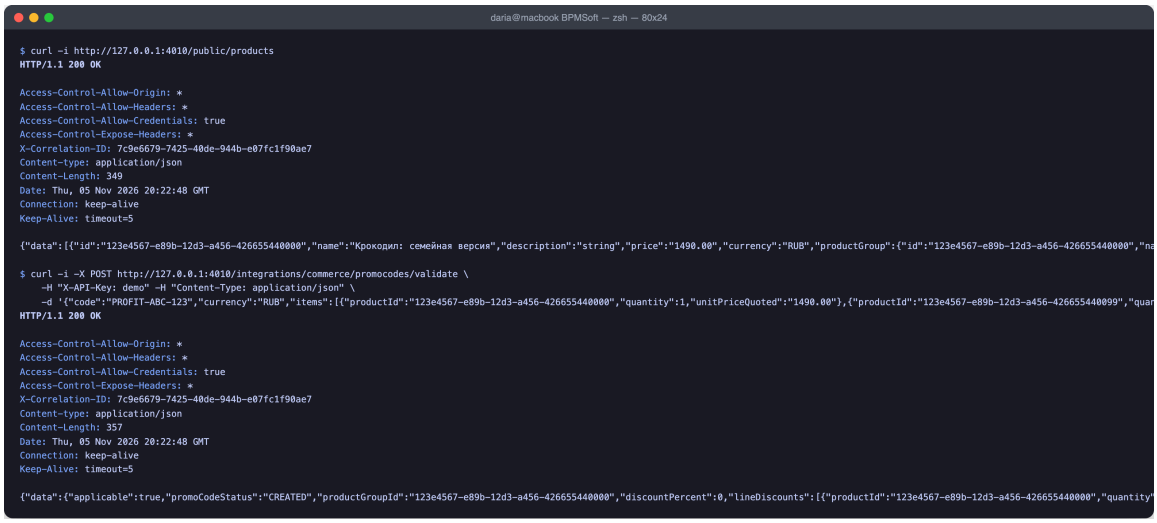


Рис. 6. Подробные результаты публичных и интеграционных вызовов.

Подтверждённые результаты smoke-теста

№	Сценарий	HTTP	Статус
1	`GET /public/products`	200	Подтверждён smoke
2	`POST /public/test-attempts`	201	Подтверждён smoke
3	`GET` попытки с `X-Attempt-Token`	200	Подтверждён smoke
4	Проверка промокода без ключа	401	Ожидаемый отказ
5	Проверка промокода с ключом	200	Подтверждён smoke
6	`POST .../purchases`	200/201	Подтверждён smoke
7	`POST .../mail/provider-events`	202	Подтверждён smoke

Таблица 7. Позитивные и базовая негативная проверка.

Всего smoke выполняет 14 шагов: позитивные маршруты из таблицы выше и 8 негативных проверок следующего раздела; полный журнал прогона — `docs/validation_log.txt`. Маршрут `/integrations/commerce/sales` присутствует в контракте и проходит валидацию схемы; отдельный happy-path в smoke **не** вызывался.

Негативные сценарии

Исходный набор проверок был в основном позитивным; smoke расширен до 14 шагов: 6 позитивных + 8 негативных. Негативные ветки исполняются через заголовок `Prefer: code=…` — Prism возвращает заданный код ответа из объявленных в спецификации, поэто-

му проверяется **форма** ответа и наличие кода в контракте, а не бизнес-логика BPMSoft. Проверка `429` дополнительно убеждается, что в ответе реально присутствует `Retry-After`.

№	Операция / ситуация	Ожидаемая проверка	Статус
N-01	Чужая товарная группа	`422` (Prefer; shape)	Исполнено в smoke
N-02	Истекший промокод	`422` (Prefer; общий shape)	Исполнено (совмещено с N-03)
N-03	Уже активированный код	`422` (Prefer; общий shape)	Исполнено (совмещено с N-02)
N-04	Повтор того же ключа + тела покупки	Исходный успех без дубля	Только контракт; mock не хранит ключи
N-05	Тот же ключ, иное тело	`409` (Prefer)	Исполнено в smoke
N-06	Повтор `eventId` webhook	`409` (Prefer)	Исполнено в smoke
N-07	Токен попытки отсутствует	`401`	Исполнено в smoke
N-07b	Чужой токен	`403`	В контракте; отдельный smoke-шаг не добавлен
N-08	Повтор email + версия	`409`	В контракте; Prefer-шаг не добавлен
N-09	Превышен лимит частоты	`429` + `Retry-After`	Исполнено в smoke
N-10	Временная недоступность	`503`	В контракте на всех ops; Prefer-шаг не добавлен
N-11	Слишком короткий `Idempotency-Key` (purchases)	`400`/`422`	Исполнено в smoke → `422`

Таблица 8. Статус негативных проверок относительно `scripts/smoke-mock.js`.

Механические проверки Prism должны дополняться интеграционными тестами настоящего сервиса: наличие статуса в спецификации не означает реальный контроль жизненного цикла, дедупликацию, соблюдение лимитов или атомарную запись покупки. Следующий шаг доказательности — прогон тех же сценариев против стенда с реальной реализацией, где `Prefer`-заголовок уже не понадобится.

Открытые вопросы и дальнейшая работа

Зафиксированные в контрактах и отчёте открытые вопросы (все помечены «проектное решение» или «предложение для согласования»):

- Продуктовое правило повторного прохождения теста: сколько попыток допустимо на email/версию в принципе — `409`/`429` ограничивают злоупотребление, но не заменяют правило.
- Частота опроса планировщика, критерий просрочки и SLA проверок сроков; конкретные TTL, число повторов и интервалы backoff.
- Применение `x-message-ttl` / `x-dead-letter-*` на реальном RabbitMQ и срок хранения DLQ (предложено 7 суток).
- Потребность в потоковом событии первой отправки для аналитики вне БД (сейчас R-2–R-4 — по `sentAt` через Insight).
- Альтернатива `CommerceApiKey`: OAuth 2.0 Client Credentials; конкретный алгоритм `X-Provider-Signature` — по документации выбранного провайдера.

- Правила сверки `/sales` с `/purchases` и обработка возвратов исторических данных — вне MVP-контракта.
- Заголовки `RateLimit-*` — после появления реализации лимитера; сейчас действует единое соглашение `Retry-After`. Предполагаемое семейство — `RateLimit-Limit`, `RateLimit-Remaining`, `RateLimit-Reset` (и `RateLimit-Policy` при нескольких политиках) по черновику IETF «RateLimit header fields for HTTP»: семейство зафиксировано и в `info.description` OpenAPI, чтобы была фактура для согласования с бизнес-заказчиком.

Валидация спецификаций и воспроизводимость

Замечание об отсутствии валидации по схеме закрыто: `swagger-parser` валидирует `openapi.yaml` по OpenAPI 3.0.3 и разрешает все `$ref`, `@asyncapi/parser` разбирает `asyncapi.yaml` без ошибок, `redocly lint` даёт дополнительную проверку структуры и компонентов. Дублирующиеся ключи YAML выявляются парсерами при чтении. В журнал пишутся версии Node.js, npm, Prism и обоих парсеров, `git HEAD` и отпечаток `git write-tree`; `scripts/run-evidence.sh` поднимает Prism и прогоняет smoke за один вызов.

```
node --version
npm --version
npm list @stoplight/prism-cli --depth=0
git rev-parse HEAD
npm run check
npm run lint:openapi
npm run mock   # терминал 1
npm run smoke  # терминал 2
```

Заключение

Разработан комплект интеграционных контрактов ProfIT: OpenAPI описывает публичные HTTP-операции и доверенные интеграции, AsyncAPI — команды и события фоновых процессов. Сопоставление с C1/C2 покрывает каталог, онлайн-тесты, промокоды, покупки, отчётность и почтовую интеграцию.

Выявленные при ревизии дефекты REST (security токена попытки, `Retry-After`, корреляция, среды/валюты/503) и AsyncAPI (TTL/DLQ в конфигурации очередей, канал `promocodes.dlx` и его потребитель, ADR планировщика и outbox-контейнер на C2, ссылки между контрактами) устранены в актуальных YAML и диаграмме. Локальная валидация и расширенный smoke зафиксированы в `docs/validation_log.txt`.

По критериям качества контрактов: полнота покрытия потоков, качество REST-контракта и согласованность с доменной моделью сохранены; безопасность контракта обеспечена переводом `X-Attempt-Token` в `securitySchemes`; асинхронный контракт усилен TTL/DLQ и зафиксированным ADR; проверяемость дополнена негативным smoke, валидацией по схеме и фиксацией версий; отчёт описывает обе спецификации.

Ограничения: Prism не исполняет доменные правила BPMSoft; часть негативов (N-08, N-10, реальный replay idempotency, `/sales` happy-path) остаётся на усмотрение доработки smoke или интеграционных тестов стенда. Развёртывание RabbitMQ с заявленными TTL/DLQ в практику не входило.

Список использованных источников

- Отчёт по практической работе № 4: ProfIT.
- Материалы предыдущих практик (C1/C2, доменная модель, бэклог E-1–E-7).
- Репозиторий команды: https://github.com/ChaoticPost/BPMSoft.
- OpenAPI Specification 3.0.3: https://spec.openapis.org/oas/v3.0.3.html.
- AsyncAPI Specification 3.0: https://www.asyncapi.com/docs/reference/specification/latest.
- AsyncAPI AMQP 0-9-1 Bindings: https://github.com/asyncapi/bindings/tree/master/amqp.
- CloudEvents Specification 1.0: https://cloudevents.io/.
- Stoplight Prism (mocking): https://docs.stoplight.io/docs/prism.
- HTTP Semantics — RFC 9110 (в т.ч. `Retry-After`, `Prefer`); Problem Details for HTTP APIs — RFC 9457.
- Журнал локальной проверки: `docs/validation_log.txt`.